RISC201 ALU: Algorithms and Floating-Point Arithmetic

src/alu.py | adders, multipliers, dividers, shifter | IEEE-754 float unit and converter

1. Overview

- **One class does all the work.** `RISC201_ALU` performs every arithmetic, logic, shift, floating-point and conversion operation of the processor, and also the internal adds of the pipeline (load/store addresses, SP plus or minus 4, the call link PC + 4).
- **Three units inside:** an integer unit (adder, multiplier, divider, barrel shifter, logic), a floating-point unit for IEEE-754 single precision, and an int/float converter for `fcvt`.
- **Selectable algorithms:** adder Kogge-Stone or ripple-carry, multiplier Booth radix-4 or shift-and-add, divider non-restoring or restoring. All give identical results; they differ in the amount of hardware work. The float unit uses the same selected adder, multiplier and divider for its significand arithmetic.
- **Every call returns its latency.** The execute stage uses it to hold the pipeline (`exBusy`) while a multi-cycle operation finishes.
- **No arithmetic exceptions.** Integer overflow wraps, divide by zero gives fixed values, and float overflow, underflow and invalid operations give Inf, zero and NaN, as IEEE-754 does by default. The only exceptions of the processor come from the stack (see the project documentation).

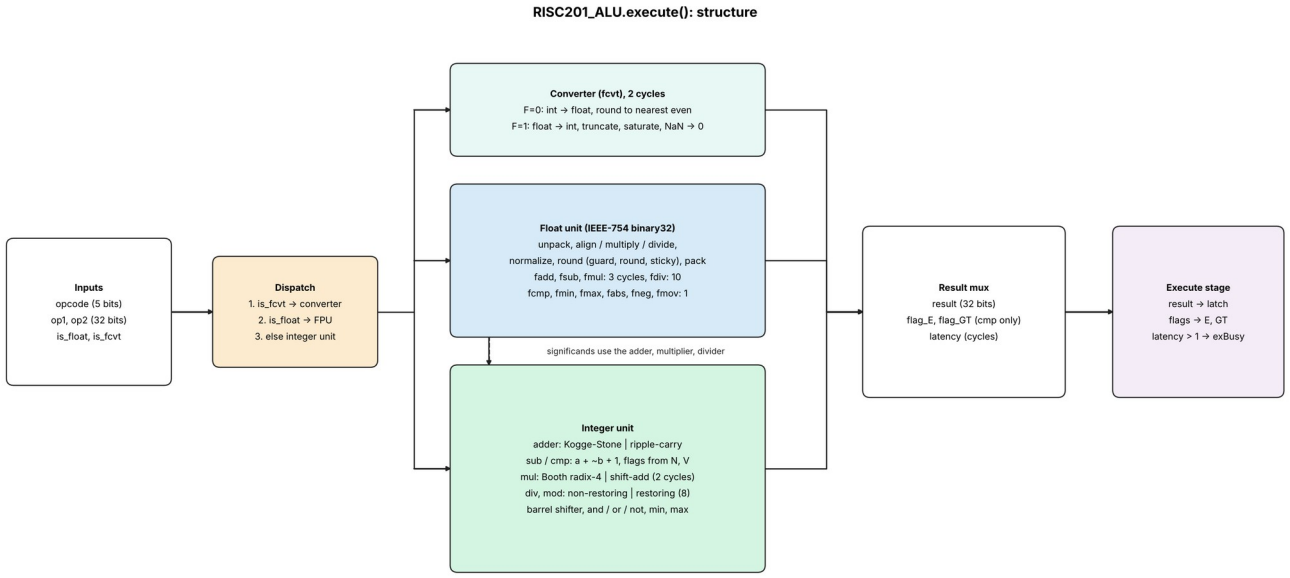


Figure 1. Structure of RISC201_ALU.execute(): dispatch to the converter, the float unit or the integer unit.

A. Operations and latencies

Opcode	Operation	Integer (F = 0)	Float (F = 1)	Cycles
00000	add	adder	fadd	1 / 3
00001	sub	adder: a + ~b + 1	fsub	1 / 3
00010	mul	Booth radix-4 or shift-and-add	fmul	2 / 3
00011	div	non-restoring or restoring divider, quotient	fdiv	8 / 10
00100	mod	divider, remainder	-	8
00101	cmp	subtract, set E and GT	fcmp	1 / 1
00110 00111 01000	and, or, not	bitwise logic	-	1
01001	mov	copy op2	fmov (copy bits)	1 / 1

Opcode	Operation	Integer (F = 0)	Float (F = 1)	Cycles
01010-01100	lsl, lsr, asr	barrel shifter	-	1
11011 11100	min, max	signed compare and select	fmin, fmax	1 / 1
11101 11110	abs, neg	absolute value, 0 - x	fabs, fneg (sign bit)	1 / 1
11111	fcvt	-	F = 0: int to float, F = 1: float to int	2

2. Interface and dispatch

A. The execute method

```
alu = RISC201_ALU(adder_mode="kogge", mul_mode="booth", div_mode="nonrestoring")
result, flag_E, flag_GT, latency = alu.execute(opcode, op1, op2, is_float=False, is_fcvt=False)
```

- `opcode` is the 5-bit instruction opcode, `op1` and `op2` are 32-bit values (for an immediate form `op2` is the immediate built by the decode stage). Both are masked to 32 bits first.
- `result` is always an unsigned 32-bit value. `flag_E` and `flag_GT` are meaningful only for `cmp` and `fcmp`; every other operation returns 0 for both.
- `latency` is the number of execute-stage cycles: 1, 2, 3, 8 or 10.
- The constructor raises `ValueError` for an unknown algorithm name.

B. Order of dispatch

- **1. Converter:** if `is_fcvt` is set (or the opcode is 11111), the converter runs; `is_float` gives the direction.
- **2. Float unit:** otherwise, if `is_float` is set, the float operation for the opcode runs.
- **3. Integer unit:** otherwise the integer operation for the opcode runs.

C. How the processor uses it

- The microinstruction decides what the ALU does. `opc` and `fopc` send the instruction's own opcode with `is_float` = 0 or 1; `cmp` and `fcmp` send opcode 00101; `i2f` and `f2i` set `is_fcvt` with direction 0 or 1; `add` and `sub` are fixed operations used for addresses, SP and the call link.
- The execute stage calls `execute` once, in the first cycle of the instruction, and keeps the result while the unit is busy for `latency - 1` more cycles.
- `alu.steps` counts the add/subtract operations done inside the multiplier and divider (`mul_addsub`, `div_addsub`), which the evaluation uses to compare the algorithms. Float operations are counted separately (`fadd_addsub`, `fmul_addsub`, `fdiv_addsub`), so the integer figures are not mixed with them.

D. Integer representation

Registers hold 32-bit patterns. Signed operations read them as two's complement with `to_signed(x)` (subtract 2^32 when bit 31 is set) and every result is masked back to 32 bits with `& 0xFFFFFFFF`. The same bit pattern can therefore mean an unsigned number, a signed number or a float; only the operation decides.

3. Adders

Both adders compute `a + b + cin` on 32 bits and return the sum and the carry out. They are used by `add`, `sub`, `cmp`, `neg` and all address calculations.

A. Ripple-carry adder

A chain of 32 full adders. Bit i adds `a_i`, `b_i` and the carry from bit i-1; its carry goes on to bit i+1, so the carry must travel through all 32 stages before the top sum bit is known.

```
for i in range(32):
    ai = (a >> i) & 1
    bi = (b >> i) & 1
    sum_bit = ai ^ bi ^ c                    # sum = a XOR b XOR carry_in
```

```
    c = (ai & bi) | (bi & c) | (ai & c)        # carry_out = majority(a, b, carry_in)
    s |= (sum_bit << i)
```

Worked example, the low 8 bits of 45 + 27 = 72 (0x2D + 0x1B = 0x48):

Bit i	a_i	b_i	carry in	sum bit	carry out
0	1	1	0	0	1
1	0	1	1	0	1
2	1	0	1	0	1
3	1	1	1	1	1
4	0	1	1	0	1
5	1	0	1	0	1
6	0	0	1	1	0
7	0	0	0	0	0

Reading the sum column from bit 7 down to bit 0 gives 01001000 = 72. **Cost:** 32 full adders; the delay grows linearly with the width (about 2 gate delays per bit, 64 in total).

B. Kogge-Stone parallel-prefix adder (default)

The carry into each bit is computed in a logarithmic number of levels instead of a chain:

- **Level 0:** for every bit, `g_i = a_i AND b_i` (this bit generates a carry) and `p_i = a_i XOR b_i` (this bit propagates an incoming carry).
- **Prefix levels:** at offsets 1, 2, 4, 8 and 16, every bit i >= offset combines its group with the group `offset` places to the right: `G = G_hi OR (P_hi AND G_lo)`, `P = P_hi AND P_lo`. After the level with offset k, `G_i` covers bits i down to i - 2k + 1.
- **After 5 levels** `G_i` is the carry out of bit i for the whole word, so `carry_into_bit(i+1) = G_i`.
- **Sum:** `s_i = p_i XOR carry_i`, using the original `p_i`.
- **Carry in:** folded into bit 0 before the prefix levels: `g_0 = g_0 OR (p_0 AND cin)`. This is how subtraction adds its +1.

```
g = [a_i & b_i for each bit]                  # level 0
p = [a_i ^ b_i for each bit]
g[0] = g[0] | (p[0] & cin)                    # fold the carry in
for step in [1, 2, 4, 8, 16]:                 # 5 prefix levels
    for i in range(step, 32):
        next_g[i] = g[i] | (p[i] & g[i - step])
        next_p[i] = p[i] & p[i - step]
c = [cin] + [g[i] for i in range(31)]         # carry into each bit
s_i = raw_p[i] ^ c[i]                         # sum
```

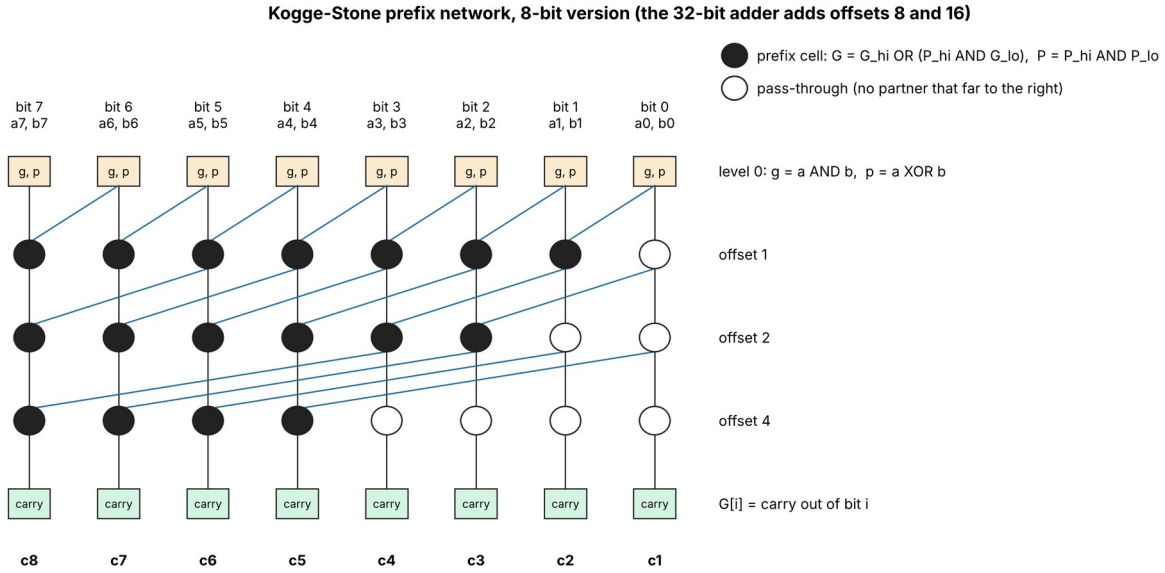


Figure 2. Kogge-Stone prefix network for 8 bits. The 32-bit adder has two more levels (offsets 8 and 16).

The same example, 0x2D + 0x1B, through the prefix levels (bit 7 on the left):

Level	G (bits 7..0)	P (bits 7..0)
g / p (level 0)	`00001001`	`00110110`
after offset 1	`00011011`	`00100100`
after offset 2	`00111111`	`00000000`
after offset 4	`00111111`	`00000000`

Carries into bits 7..0: `01111110`. Sum = original p XOR carries = `00110110` XOR `01111110` = `01001000` = 72. Already after offset 2 every G is final here, because no carry chain in this example is longer than 4 bits; the hardware always runs all levels.

Cost: 5 prefix levels plus the g/p and sum levels (7 levels, about 14 gate delays) and 129 prefix cells for 32 bits (31 + 30 + 28 + 24 + 16). It trades more cells for a much shorter critical path than ripple-carry.

C. Overflow flag

- Kogge-Stone: `V = carry_out(bit 31) XOR carry_into(bit 31)`: signed overflow happens exactly when these two carries differ.
- Ripple-carry: `V = ((a XOR s) AND (b XOR s)) >> 31`: overflow when both operands have the same sign and the sum has the other sign.
- Both formulas give the same V. It is not stored in a register; `cmp` uses it (section 4).

4. Subtraction and comparison

A. Subtraction

`a - b` is computed as `a + (NOT b) + 1` on the same adder: the inverted operand goes in as b and the +1 as the carry in. `neg x` is `0 - x` on the same path.

B. Compare (cmp) and the flags

`cmp` subtracts and sets the flags from the difference, using the sign bit N and the overflow V of the subtraction:

```
diff, _, v = self._sub(op1, op2)
n = (diff >> 31) & 1                # sign of the difference
is_less = n ^ v                     # signed a < b
flag_e  = 1 if diff == 0 else 0
flag_gt = 1 if (diff != 0 and not is_less) else 0
```

`N XOR V` is the correct signed less-than even when the subtraction overflows; looking at N alone would give the wrong answer in the last two rows:

a	b	a - b (32 bits)	N	V	N XOR V (a < b)	E	GT
5	7	0xFFFFFFFE	1	0	1	0	0
7	5	0x00000002	0	0	0	0	1
9	9	0x00000000	0	0	0	1	0
-3	2	0xFFFFFFFB	1	0	1	0	0
-2147483648	1	0x7FFFFFFF	0	1	1	0	0
2147483647	-1	0x80000000	1	1	0	0	1

5. Multipliers

A. Signed and unsigned give the same low 32 bits

`mul` keeps the low 32 bits of the product. These bits are the same whether the operands are read as signed or unsigned (two's complement multiplication modulo 2^32), so neither multiplier needs a separate signed mode.

B. Booth radix-4 multiplier (default)

Booth recoding looks at the multiplier two bits at a time, plus the highest bit of the previous pair (a 0 is appended below bit 0). Each overlapping group of 3 bits becomes one digit in {-2, -1, 0, +1, +2}, so 16 digits cover 32 bits and each digit needs at most one add or subtract of the multiplicand (or of twice it, which is a shift).

Bits (2i+1, 2i, 2i-1)	Digit	Partial product
000, 111	0	nothing
001, 010	+1	+ multiplicand
011	+2	+ 2 x multiplicand (shift left 1)
100	-2	- 2 x multiplicand
101, 110	-1	- multiplicand

```
padded_r = (r << 1)                          # append b_-1 = 0
for i in range(16):
    triplet = (padded_r >> (2 * i)) & 0b111
    d = {0b001: 1, 0b010: 1, 0b011: 2, 0b100: -2, 0b101: -1, 0b110: -1}.get(triplet, 0)
    acc += (d * m) << (2 * i)                # partial product, weight 4^i
return acc & MASK32
```

Worked example: 25 x (-7). The multiplier -7 is `0xFFFFFFF9`.

Digit i	Bits	Digit	Contribution d x 25 x 4^i
0	`010`	1	25
1	`100`	-2	-200
2	`111`	0	0
3 to 15	`111`	0	0

25 - 200 = -175 = `0xFFFFFF51`. Only 2 of the 16 digits needed an add or subtract: the long run of 1 bits in a negative number costs nothing, which is the point of Booth recoding.

C. Shift-and-add multiplier

The textbook method: for every 1 bit of the multiplier, add the multiplicand shifted to that bit position.

```
for i in range(32):
    if (b >> i) & 1:
        acc = (acc + (a << i)) & MASK32
```

Bit i of 7	Added (25 << i)	Accumulator
0	25	25
1	50	75
2	100	175

25 x 7 = 175 in 3 additions. For a negative multiplier such as -7, all 32 bits except two are 1, so shift-and-add needs 30 additions where Booth needs 2.

D. Comparison

	Booth radix-4	Shift-and-add
Steps	16 digits	32 bits
Add/sub per multiply (random 32-bit operands)	12.0 on average, at most 16	16.1 on average, up to 32
Negative or large multipliers	cheap (runs of 1s cost nothing)	expensive (one add per 1 bit)
Small positive multipliers (2..10)	about the same	slightly fewer adds
Hardware	digit recoder, partial products of +-1x / +-2x	simple AND gates and adders

6. Dividers

A. What both dividers share

- **Division by zero:** no exception. `div` returns `0xFFFFFFFF` and `mod` returns the dividend.
- **INT_MIN / -1:** the true quotient 2^31 does not fit, so the result is INT_MIN with remainder 0.
- **Signs:** the algorithm divides the magnitudes; then the quotient is negated if the signs differ, and the remainder takes the sign of the dividend. This is truncating division (the quotient rounds toward zero), the same as C and Java.

Division	Quotient	Remainder
13 / 4	3 (0x00000003)	1
-13 / 4	-3 (0xFFFFFFFD)	-1
13 / -4	-3 (0xFFFFFFFD)	1
-13 / -4	3 (0x00000003)	-1
7 / 0	-1 (0xFFFFFFFF)	7
-2147483648 / -1	-2147483648 (0x80000000)	0

B. Non-restoring divider (default)

Registers: A (33 bits, signed partial remainder) and Q (32 bits, starts as the dividend, ends as the quotient). Each of the 32 steps:

- Shift the pair [A, Q] left by one bit (the top bit of Q moves into A).
- If A is negative, **add** the divisor; otherwise **subtract** it. This is the key idea: a negative A is not restored, the next step corrects it by adding instead of subtracting.
- If A is now non-negative, set the new low bit of Q to 1, otherwise leave it 0.
- After 32 steps, if A is negative, add the divisor once (final correction). A is the remainder, Q the quotient.

Worked example: the same loop with 4 bits instead of 32, for 13 / 4 (dividend 1101, divisor V = 4):

Step	Operation	A after the step	Q
1	A - V	-3	`1010`
2	A + V	-1	`0100`
3	A + V	2	`1001`
4	A - V	1	`0011`

Result: Q = 0011 = 3, A = 1 = remainder, no final correction needed (A is already non-negative). Every step costs exactly one add or subtract: 32 for a 32-bit division, plus at most one correction.

C. Restoring divider

The classical method. Each step shifts, **subtracts** the divisor, and if the partial remainder became negative **adds it back** (restores it) and writes a 0 quotient bit; otherwise it keeps the result and writes a 1.

Step	Operation	A after the step	Q
1	A - V < 0: restore A + V	1	`1010`
2	A - V < 0: restore A + V	3	`0100`
3	A - V >= 0: keep, q = 1	2	`1001`
4	A - V >= 0: keep, q = 1	1	`0011`

Same result, 3 remainder 1, but the two steps that went negative needed a second operation. In the full 32-bit divider 13 / 4 takes 62 add/subtract operations against 32 for the non-restoring divider, because most of the early steps of a small dividend go negative.

D. Comparison

	Non-restoring	Restoring
Operations per step	always 1 (add or subtract)	1 or 2 (subtract, maybe restore)
Add/sub per division (random operands)	32.6 on average, at most 33	57.6 on average, up to 64
Extra step	one final correction if A < 0	none
Hardware	adder/subtractor controlled by the sign of A	subtractor plus a restore path or register

7. Shifter, logic and other integer operations

A. Barrel shifter

Only the low 5 bits of the shift amount are used (0 to 31). In hardware the shifter is 5 levels of multiplexers that shift by 16, 8, 4, 2 and 1; the levels whose bit is set in the amount are active. `lsr` fills with zeros, `asr` copies the sign bit. The model computes the same result with one shift operation.

Operation	Active levels	Result
lsl 0x00000001, 13	8, 4, 1	0x00002000
lsr 0x80000000, 4	4	0x08000000
asr 0x80000000, 4	4	0xF8000000
asr 0xFFFFFF80, 4	4	0xFFFFFFF8
lsl 0x00000001, 33	1	0x00000002

The last row shows the 5-bit masking: a shift by 33 is a shift by 1.

B. Logic, min, max, abs, neg

Operation	Result	As signed
min -8, 8	0xFFFFFFF8	-8
max -8, 8	0x00000008	8
abs -8	0x00000008	8
abs INT_MIN	0x80000000	-2147483648
neg 5	0xFFFFFFFB	-5
not 0x0F	0xFFFFFFF0	-16

- `and`, `or`, `not` are bitwise on 32 bits. `min` and `max` compare as signed numbers.
- `abs INT_MIN` stays INT_MIN, because +2^31 does not fit in 32 signed bits (the usual two's complement behaviour).
- `neg` uses the adder: `0 + NOT x + 1`.

8. Floating-point numbers

A. The binary32 format

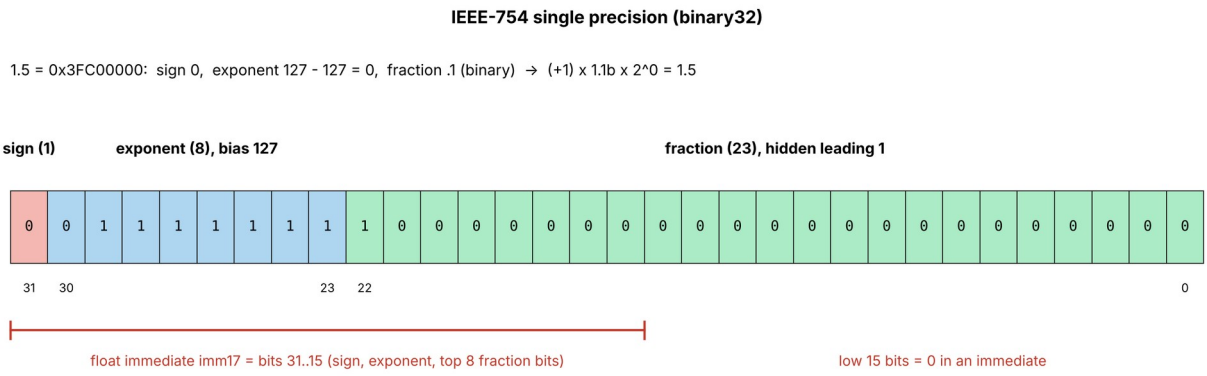


Figure 3. IEEE-754 single precision fields, the value 1.5 and the 17 bits kept in a float immediate.

A normal number is `(-1)^sign x 1.fraction x 2^(exponent - 127)`. Integers and floats share the 16 registers; a register holds only the 32 bits, and the instruction decides whether they are read as an integer or a float.

Exponent field	Fraction	Meaning	In RISC201
0	0	+0 or -0	zero
0	not 0	subnormal (very small)	flushed to signed zero, as input and as result
1 to 254	any	normal number	computed with round to nearest even
255	0	+Inf or -Inf	result of overflow or x / 0
255	not 0	NaN (not a number)	every NaN result is 0x7FC00000

B. How the float unit computes a result

The float unit works directly on the bits of the operands, in the same steps as a hardware unit. It does not use the host computer's floating-point arithmetic: every step below is integer and bit operations, and the significand arithmetic goes through the ALU's own adder, multiplier and divider.

- **1. Unpack.** `unpack_float` splits the 32 bits into the sign, the 8-bit exponent field and the 24-bit significand `1.fraction` (the hidden 1 is made explicit, so the significand is an integer from 2^23 to 2^24 - 1). An exponent field of 0 is a zero: the fraction of a subnormal is ignored (flush to zero). An exponent field of 255 is Inf or NaN.

- **2. Special values first.** NaN operands, Inf - Inf, Inf x 0, 0 / 0 and Inf / Inf give the canonical NaN 0x7FC00000; other cases with Inf or zero give their fixed result (section G) without any arithmetic.
- **3. Operate on the significands.** Align and add or subtract (fadd, fsub), multiply (fmul) or divide (fdiv), keeping extra low bits for rounding (part C).
- **4. Normalize.** A leading-one detector finds the highest 1 of the result and the result is shifted so that 24 significand bits remain, with the exponent adjusted by the same amount.
- **5. Round and pack.** Round to nearest, ties to even (part C), then check the range: a result too large becomes +Inf or -Inf, a result below the smallest normal number (2^-126) becomes a signed zero. Finally the sign, exponent and 23 fraction bits are packed into 32 bits.

```
def unpack_float(bits):
    sign, exp, frac = bits >> 31, (bits >> 23) & 0xFF, bits & 0x7FFFFF
    if exp == 0:    return sign, 0, 0                  # zero (a subnormal is flushed)
    if exp == 0xFF: return sign, 0xFF, frac            # Inf (frac 0) or NaN
    return sign, exp, (1 << 23) | frac                 # 24-bit significand 1.f
```

Steps 4 and 5 are shared by all operations and by the int-to-float converter, in `_round_pack(sign, m, scale)`, which packs the value `m x 2^scale`. The exponent is a small 8-bit number, so its additions and subtractions are done with ordinary integer arithmetic.

C. Rounding with guard, round and sticky bits

After normalization the result keeps three bits below its 24 significand bits:

- **Guard (G):** the first bit below the significand, worth half a unit in the last place (ulp).
- **Round (R):** the next bit, worth a quarter ulp.
- **Sticky (S):** the OR of every bit below R. Whenever a shift throws bits away, `_shr_sticky` ORs them into bit 0, so S records only whether anything non-zero was lost.

```
guard, rnd, sticky = (m >> 2) & 1, (m >> 1) & 1, m & 1
sig = m >> 3                                          # the 24 significand bits
if guard and (rnd or sticky or (sig & 1)):            # above half, or exactly half and odd
    sig, _, _ = self.alu._add(sig, 1)                 # round up on the selected adder
if sig == 1 << 24:                                    # 1.111..1 + 1 ulp = 10.000..0
    sig >>= 1; exp += 1
```

G R S	Distance from the lower value	Action
0 x x	less than half an ulp	keep (round down)
1 0 0	exactly half an ulp (a tie)	round to the even value: up only if the last significand bit is 1
1 1 x or 1 x 1	more than half an ulp	round up

- **Why three bits are enough.** For an add the operands are aligned with a sticky shift, so the sum differs from the exact sum by less than one unit of the S position and stays on the same side of every rounding boundary. A subtraction can cancel the top bit and need a shift left by one, which turns R into G and keeps S below it; a larger shift left only happens when the exponents differ by 0 or 1, and then no bits were lost. A product or quotient is exact (or has an exact sticky bit from the remainder) before rounding. So every operation returns exactly the binary32 value IEEE-754 defines.
- **Below the normal range.** If the exponent of the result is 0 or less, the significand is first shifted right with sticky to the position a subnormal would have, and rounded there. If that rounds up to 2^-126 the result is the smallest normal number; otherwise it is a subnormal and is flushed to a signed zero.
- **Overflow.** If the rounded exponent reaches 255 the result is +Inf or -Inf.

D. The operations step by step

Operation	Significand step	Unit used	Cycles
fadd, fsub	order the operands by magnitude; shift the smaller significand right by the exponent difference (sticky shift); add if the signs are equal, else subtract the smaller from the larger. fsub is fadd with the sign of b flipped. An exact zero difference gives +0	selected adder (one 32-bit pass)	3
fmul	multiply the two 24-bit significands into a 48-bit product; exponent = ea + eb - 127; the sign is sa XOR sb	selected multiplier: Booth radix-4 (13 digits) or shift-and-add (24 bits); each partial product is added as two chained passes of the 32-bit adder (64-bit accumulator)	3
fdiv	long division of the significands, 27 quotient bits (24 + guard + round + one spare because ma / mb lies between 0.5 and 2); sticky = remainder not zero; exponent = ea - eb + 127	selected divider: non-restoring or restoring, every step on the selected adder	10
fcmp, fmin, fmax	compare as sign-magnitude integers (no arithmetic)	comparator	1
fabs, fneg, fmov	clear, flip or copy the sign bit	bit operations	1

Booth for the significand product. The multiplier significand is unsigned, so it is read as a 26-bit signed number whose top two bits are 0. That gives 13 Booth digits instead of 16; a negative partial product is added as its complement with a carry in of 1, as in the integer subtraction.

The two dividers on significands. Restoring: subtract the divisor; if the result is negative keep the old remainder and write a 0, otherwise keep the result and write a 1; shift left. Non-restoring: start with `ma - mb`; while the partial remainder is non-negative, the next step subtracts the divisor, while it is negative the next step adds it; a final add corrects a negative remainder. Both produce the same 27 quotient bits.

Cycles and timing. The cycle counts are the timing figures of the design and are reported by the ALU; the model does not derive them from the number of steps. A real 10-cycle float divider would produce several quotient bits per cycle (for example a radix-4 SRT divider); the model works out the same quotient one bit at a time.

E. Operation reference

Instruction	What it does
`fadd`, `fsub`, `fmul`	IEEE-754 binary32 result, round to nearest even, then flush-to-zero
`fdiv`	Same; for a zero divisor: 0/0 and NaN/0 give NaN, x/0 gives Inf with sign = sign(x) XOR sign(divisor)
`fcmp`	E = 1 if equal, GT = 1 if greater; if either operand is NaN both flags are 0 (unordered), so `beq` and `bgt` are not taken; -0 equals +0
`fmin`, `fmax`	the smaller / larger value; if one operand is NaN the other is returned
`fabs`	clears the sign bit (`bits AND 0x7FFFFFFF`)
`fneg`	flips the sign bit (`bits XOR 0x80000000`)
`fmov`	copies the bits of a register or the float immediate

F. Worked examples

Value	Bits	Sign	Exponent field	Exponent	Fraction (23 bits)
1.5	0x3FC00000	0	127	0	`10000000000000000000000`

Value	Bits	Sign	Exponent field	Exponent	Fraction (23 bits)
2.25	0x40100000	0	128	1	00100000000000000000
3.75	0x40700000	0	128	1	11100000000000000000
8.4375	0x41070000	0	130	3	00001110000000000000

1.5 + 2.25 (fadd). Unpack: 2.25 has exponent field 128 and significand `0x900000` (1.001b), 1.5 has 127 and `0xC00000` (1.1b). 2.25 is larger, so it goes first. With the three extra bits the significands become `0x4800000` and `0x6000000`; the second is shifted right by the exponent difference 1 to `0x3000000` (no bits lost, S = 0). One pass of the adder: `0x4800000 + 0x3000000 = 0x7800000`. The leading 1 is already at bit 26, G R S = 000, so no rounding; significand `0xF00000` = 1.111b, exponent field 128: `0x40700000` = 3.75.

3.75 x 2.25 (fmul). Significands `0xF00000` (1.111b) and `0x900000` (1.001b). Booth recoding of `0x900000` gives only three non-zero digits: +1 at digit 10, -2 at digit 11 and +1 at digit 12 (2^20 - 2^23 + 2^24 = 2^23 + 2^20), so the product takes 3 adder operations: `0x870000000000`, a 48-bit number. Normalizing shifts it right by 21 bits (all of them 0, so S = 0) to significand `0x870000` = 1.0000111b; the exponent is 128 + 128 - 127 + 1 = 130 (the extra 1 because the product of two significands between 1 and 2 reached 2). Result `0x41070000` = 8.4375.

1 / 3 (fdiv). Significands `0x800000` and `0xC00000`. Because 1.0 < 1.5 the first quotient bit is 0, and the 27 quotient bits are `0x2AAAAAA` = 1010...10b (26 significant bits); the remainder is not zero, so S = 1. With the sticky bit appended this is `0x5555555`: significand `0xAAAAAA`, G = 1, R = 0, S = 1, more than half an ulp, so it rounds up to `0xAAAAAB`. Exponent field 127 - 128 + 127 - 1 = 125: `0x3EAAAAAB` = 0.33333334. The non-restoring divider needs 28 add/subtract operations for this (27 steps and one final correction), the restoring divider 41.

Rounding ties. Three additions whose exact result lies exactly halfway between two binary32 values:

Operation	Exact result	G R S	Result
1.0 + 2^-24	1 + half an ulp	1 0 0, last bit 0	`0x3F800000` = 1.0 (stays even)
(1 + 2^-23) + 2^-24	1 + 1.5 ulp	1 0 0, last bit 1	`0x3F800002` = 1 + 2^-22 (rounds up to even)
1.0 - 1.5 x 2^-24	1 - 1.5 ulp of the range below 1	after a 1-bit left shift: 1 0 0, last bit 0	`0x3F7FFFFE` (stays even)

In the last row the subtraction `0x4000000 - 0x6` = `0x3FFFFFA` has its leading 1 at bit 25, one place too low, so normalization shifts it left by one and decreases the exponent; the low bits 100 then form G R S.

Operation	a	b	Result bits	Result value
1.5 + 2.25	0x3FC00000	0x40100000	0x40700000	3.75
1.5 - 2.25	0x3FC00000	0x40100000	0xBF400000	-0.75
3.75 x 2.25	0x40700000	0x40100000	0x41070000	8.4375
1.0 / 3.0	0x3F800000	0x40400000	0x3EAAAAAB	0.3333333432674408
0.1 + 0.2	0x3DCCCCCD	0x3E4CCCCD	0x3E99999A	0.30000001192092896

`1.0 / 3.0` and `0.1 + 0.2` show rounding: 1/3 has no exact binary32 value, and 0.1 and 0.2 are themselves rounded, so their sum is 0.30000001 in single precision.

G. Special values

Operation	Result bits	Value
big x big (1.7e38 x 1.7e38)	0x7F800000	inf
tiny x tiny (1.18e-38 squared)	0x00000000	0.0
0.0 / 0.0	0x7FC00000	nan

Operation	Result bits	Value
1.0 / 0.0	0x7F800000	inf
1.0 / -0.0	0xFF800000	-inf
-1.0 / 0.0	0xFF800000	-inf
Inf - Inf	0x7FC00000	nan
NaN + 1.0	0x7FC00000	nan
subnormal 0x00000001 + 0.0	0x00000000	0.0
fabs(-2.5)	0x40200000	2.5
fneg(2.5)	0xC0200000	-2.5
fmin(NaN, 3.0)	0x40400000	3.0

H. Float compare

Comparison	E	GT	Note
2.0 vs 1.0	0	1	greater
1.0 vs 1.0	1	0	equal
-0.0 vs 0.0	1	0	the two zeros are equal
NaN vs 1.0	0	0	unordered: no flag set
subnormal vs 0.0	1	0	the subnormal is flushed to zero first

I. Float immediates

An instruction has room for 17 immediate bits, so a float immediate keeps the top 17 bits of the binary32 value (sign, exponent and the top 8 fraction bits) and the low 15 bits become 0: `imm = imm17 << 15`. Values with at most 9 significant binary digits are exact; others are truncated, and the assembler prints a warning. A full-precision constant can be loaded with `ld`.

Written	Exact bits	imm17	Bits used	Value used	
1.5	0x3FC00000	0x07F80	0x3FC00000	1.5	exact
0.25	0x3E800000	0x07D00	0x3E800000	0.25	exact
1000.0	0x447A0000	0x088F4	0x447A0000	1000.0	exact
0.1	0x3DCCCCCD	0x07B99	0x3DCC8000	0.099853515625	truncated
3.14159	0x40490FD0	0x08092	0x40490000	3.140625	truncated

9. Conversion: fcvt and fcvti

A. Integer to float (fcvt, F = 0)

The register is read as a signed integer. The sign is taken from bit 31 and the magnitude is formed (a negative value is negated on the adder; INT_MIN gives 2^31, which still fits as an unsigned number). The magnitude then goes through the same normalize, round and pack step as the arithmetic, `_round_pack(sign, magnitude, 0)`: the leading-one detector sets the exponent and anything beyond 24 significant bits is rounded with G, R and S. Integers up to 2^24 = 16,777,216 are exact; above that only even numbers, then multiples of 4, and so on, can be represented, so the value is rounded.

B. Float to integer (fcvti, F = 1)

- NaN gives 0.

- Unpack the value; the unbiased exponent e = exponent field - 127 says where the binary point is in the 24-bit significand.
- If e >= 31 the magnitude is at least 2^31: +Inf and large positive values give INT_MAX `0x7FFFFFFF`, -Inf and large negative values give INT_MIN `0x80000000`. Zero, subnormals and values with e < 0 (magnitude below 1) give 0.
- Otherwise the barrel shifter moves the significand so that only the integer part remains: right by 23 - e bits (the fraction bits fall off, which is truncation toward zero) or left by e - 23 bits for e from 24 to 30. A negative value is then negated on the adder. So 3.99 gives 3 and -3.99 gives -3.

C. Examples

Direction	Input	Result bits	Result
int to float	7	0x40E00000	7.0
int to float	-7	0xC0E00000	-7.0
int to float	16777216	0x4B800000	16777216.0
int to float	16777217	0x4B800000	16777216.0
int to float	2147483647	0x4F000000	2147483648.0
float to int	3.99	0x00000003	3
float to int	-3.99	0xFFFFFFFD	-3
float to int	0.5	0x00000000	0
float to int	3000000000.0	0x7FFFFFFF	2147483647
float to int	-3000000000.0	0x80000000	-2147483648
float to int	+Inf	0x7FFFFFFF	2147483647
float to int	NaN	0x00000000	0

16777217 becomes 16777216.0: it lies exactly halfway between two binary32 values, and the tie goes to the one with an even last bit. 2147483647 becomes 2147483648.0 for the same reason.

10. Latency and work done

Operation	Cycles	Add/sub steps (default algorithm)
add, sub, cmp, logic, shifts, mov, min, max, abs, neg, addresses	1	one adder pass where needed
mul	2	Booth: 0 to 16, 12.0 on average
div, mod	8	non-restoring: 32 or 33
fadd, fsub	3	1 (rounding up uses one more adder pass)
fmul	3	Booth: at most 13, 10.0 on average (shift-and-add: up to 24, 12.5 on average)
fdiv	10	non-restoring: 27 or 28 (restoring: 40.4 on average, up to 53)
fcvt, fcvti	2	- (normalize and round, or shift; a negative value is negated on the adder)
fcmp, fmin, fmax, fabs, fneg, fmov	1	-

The cycle counts are fixed by the design documents and do not change with the algorithm: a hardware unit is built to finish in those cycles, and the algorithm decides how much work (and hardware) that takes. The evaluation report compares the work: Kogge-Stone needs about 7 logic levels against 32 carry stages, Booth about 25% fewer add/subtract steps than shift-and-add, and the non-restoring divider about 43% fewer than the

restoring one. The float averages in the table above are for random normal operands; they are counted under their own keys in `alu.steps`.

11. Verification and corrections

A. Tests

- `tests/test_alu.py` compares `RISC201_ALU` with the independent reference arithmetic in `tests/pipeline_sim.py`: 80,000 random integer and float operations for each of the 8 algorithm combinations (640,000 in total), mixed with special values (0, -1, INT_MIN, INT_MAX, +-Inf, NaNs with payloads, subnormals, the largest and smallest normal floats). Every result matches, and every latency matches the design documents.
- The four multiplier and divider functions are also checked directly against exact arithmetic, and all 17 example programs give the same results with all 8 combinations.
- The bit-level float unit was checked against the earlier float unit, which rounded through the host computer's double precision (correct for these operations, see 11B), as an independent oracle: 25,000 random operand pairs through fadd, fsub, fmul, fdiv, fmin, fmax, fcmp and both conversions for each of the 8 algorithm combinations, plus about 35,000 targeted cases (rounding ties, results next to the smallest normal number and next to overflow, cancellation with close exponents) and about 6,000 conversion edge cases. Every result is identical, bit for bit.

B. Corrections made during verification

Testing the first version of the float unit against the reference found these problems, which are now fixed (marked `FIX` in the code):

Problem	Correction
A result larger than binary32 (for example 1.7e38 x 1.7e38) crashed the simulator with OverflowError	returns +Inf or -Inf, as IEEE-754 requires
`fcvti` of +Inf or -Inf crashed (int() of infinity)	the range is checked first; Inf saturates to INT_MAX or INT_MIN
Subnormal inputs and results were kept, although the design flushes them to zero	flushed to signed zero on input and output
NaN results kept random payloads and signs	every NaN result is the canonical 0x7FC00000
`fdiv`: NaN / 0 returned -Inf, and a divisor of -0.0 gave the wrong sign of Inf	NaN / 0 gives NaN; the sign of Inf is sign(x) XOR sign(divisor)
The float unit computed results with the host computer's double-precision arithmetic and then rounded them. The results were correct, but the model did not show the hardware algorithm, and the selected adder, multiplier and divider had no effect on float operations	rewritten as the bit-level datapath of section 8 (unpack, align, significand arithmetic on the selected units, normalize, round with G R S, pack), checked bit for bit against the earlier version

The integer part (adders, multiplier, divider, shifter, logic) matched the reference from the start. The conversion helpers `bits_to_float` and `float_to_bits` are still in `alu.py`, but only for printing values in the debugger and the tests; the float unit does not use them.

12. Using the ALU

A. From Python

```
import sys; sys.path.insert(0, "src")
from alu import RISC201_ALU, float_to_bits, bits_to_float

alu = RISC201_ALU("kogge", "booth", "nonrestoring")
print(alu.execute(0b00010, 25, -7 & 0xFFFFFFFF))          # mul: (4294967121, 0, 0, 2) = -175
r = alu.execute(0b00000, float_to_bits(1.5), float_to_bits(2.25), is_float=True)
print(hex(r[0]), bits_to_float(r[0]), r[3])               # 0x40700000 3.75 3
```

```
print(alu.execute(0b00101, 5, 7))                    # cmp: (0, 0, 0, 1), 5 < 7
print(alu.steps)                                     # {'mul_addsub': 2, 'fadd_addsub': 1}
```

B. From the simulator and the CLI

- Command line: `python src/cli.py program.asm --adder ripple --mul shiftadd --div restoring`.
- Inside the CLI: `alu` shows the current algorithms and the add/subtract steps so far; `alu adder kogge|ripple`, `alu mul booth|shiftadd`, `alu div nonrestoring|restoring` change them (the program restarts).
- `python evaluate.py` runs every program with all 8 combinations and writes the comparison to `docs/evaluation_report.md`.